



DISEÑO DE BASE DE DATOS (SI400)
EXAMEN PARCIAL
2022-2

Sección: CC41 / CC42 / CC43 / CC44 / CC45 / CC46 / CC47
SI35 / SV42 / SV43 / WS42 / WS43 / WX41 / WX42

Profesores: Félix Corrales, Rosa Andrea
Mayta Guillermo, Jorge Luis
Morales Arévalo, Juan Carlos
Navarrete Vilca, Elio Jefferson
Noriega Meléndez, Julio Manuel
Ocampo Tello, Ernesto
Requejo Chaname, Walter Juan
Sáenz Egusquiza, Felicita Delia

Duración: 170 minutos

Indicaciones:

1. El examen consta de una (1) pregunta sobre un caso de estudio, y tendrá 170 minutos para resolverla.
2. El enunciado de la pregunta se encuentra en el archivo
upc-pre-202202-si400-examen-parcial.pdf
3. La solución deberá adjuntarse en un documento Word con la siguiente nomenclatura:
upc-pre-202202-si400-examen-parcial-codigoalumno.docx
4. Cada examen cuenta con un equipo académico, el cual estará conectado durante los primeros **15 minutos del examen**.
5. El alumno debe dedicar los primeros 15 minutos a revisar las preguntas del examen y de presentarse alguna duda enviar un correo al(los) profesor(es)

Todas las secciones:

Mayta Guillermo, Jorge Luis | pcsijmay@upc.edu.pe

6. De no recibir respuesta del equipo académico, o tener algún inconveniente adicional pasado los primeros 15 minutos, puede comunicarse con los siguientes profesores de acuerdo a su sección:

Secciones CC41 / CC42 / CC43 / CC44 / CC45 / CC46 / CC47

Requejo Chaname, Walter Juan | pcsiwreq@upc.edu.pe

Secciones SI35 / SV42 / SV43 / WS42 / WS43 / WX41 / WX42

Delgado Vite, Jorge Luis | pcsijdev@upc.edu.pe

7. El asunto del correo debe incluir el código de la sección a la cual pertenece el alumno.
8. Los profesores en mención solo recibirán correos provenientes de las cuentas UPC, **de ninguna manera se recibirán correos de cuentas públicas**.
9. Ante problemas técnicos, debe de forma obligatoria adjuntar evidencias de este, como capturas de pantalla, videos, fotos, etc. Siendo requisito fundamental que, en cada evidencia se pueda apreciar claramente la fecha y hora del sistema operativo del computador donde el alumno está rindiendo el examen.
10. Los problemas técnicos se recibirán como máximo 15 minutos culminado el examen.

Caso de estudio: DevSecOpsTalks.pe

La carrera de Ingeniería de Software de la Universidad Peruana de Ciencias Aplicadas (UPC) ha tomado la iniciativa de organizar un evento nacional gratuito de charlas virtuales en el área de DevSecOps, de modo tal que ha iniciado su proceso de llamado a propuestas de charlas (Call for Talk Proposals). Los ponentes de las charlas aceptadas serán docentes con grado académico de maestro o doctor, y deben dar cátedra en universidades públicas y/o privadas del Perú. Por lo cual se desea crear una aplicación que cuente con las siguientes características:

- El sistema debe permitir el registro de un docente. El registro del docente debe incluir nombres, apellidos, edad, grado académico y la universidad a la cual representa.
- El sistema debe permitir el registro de una universidad. El registro de la universidad debe incluir nombre y la ciudad de procedencia.
- El sistema debe permitir el registro de una propuesta de una charla por parte de un docente, no existe límite para que un docente envíe diferentes propuestas. El registro de una propuesta debe incluir también el nombre de la charla y una descripción de la misma.
- El sistema debe permitir registrar grupos revisores, los cuales están conformados por tres (3) docentes con grado de doctor de diferentes universidades del país y tienen como responsabilidad la revisión de las propuestas que se les asignen, a fin de determinar si son aceptadas o rechazadas. Cada grupo puede revisar más de una propuesta y un docente solo puede ser parte de un grupo revisor.
- El sistema debe permitir consultar en todo momento el seguimiento del estado (Recibida, Rechazada, En Revisión, Aceptada) de una propuesta de una charla.
- Las charlas aceptadas serán categorizadas por los revisores como (i) genéricas que cubre todos los aspectos de DevSecOps o (ii) especializadas en Desarrollo (Dev), Seguridad (Sec) u Operaciones (Ops).
- Las charlas aceptadas tienen una fecha, hora, duración y capacidad máxima de asistentes.
- El sistema debe permitir el registro de alumnos a las charlas. No existe un número límite de charlas a las cuales un alumno pueda inscribirse, sin embargo, no debe haber cruce de horarios.

(20 p.) Sobre el caso expuesto se pide realizar lo siguiente:

- Diseñar el modelo físico de base de datos, identificando las principales tablas y sus respectivos atributos, las relaciones entre las tablas, las llaves primarias y llaves secundarias.
- Aplicar las formas normales (1FN, 2FN y 3FN) al modelo físico de base de datos realizado.
- Escribe sentencias DDL que permiten la creación de las bases de datos, las tablas y las relaciones de las tablas.
- Escribe sentencias DML que permiten:
 - Mostrar el nombre de la universidad con la mayor cantidad de propuestas recibidas.
 - Mostrar el nombre de la charla con la mayor cantidad de alumnos inscritos.

Nota:

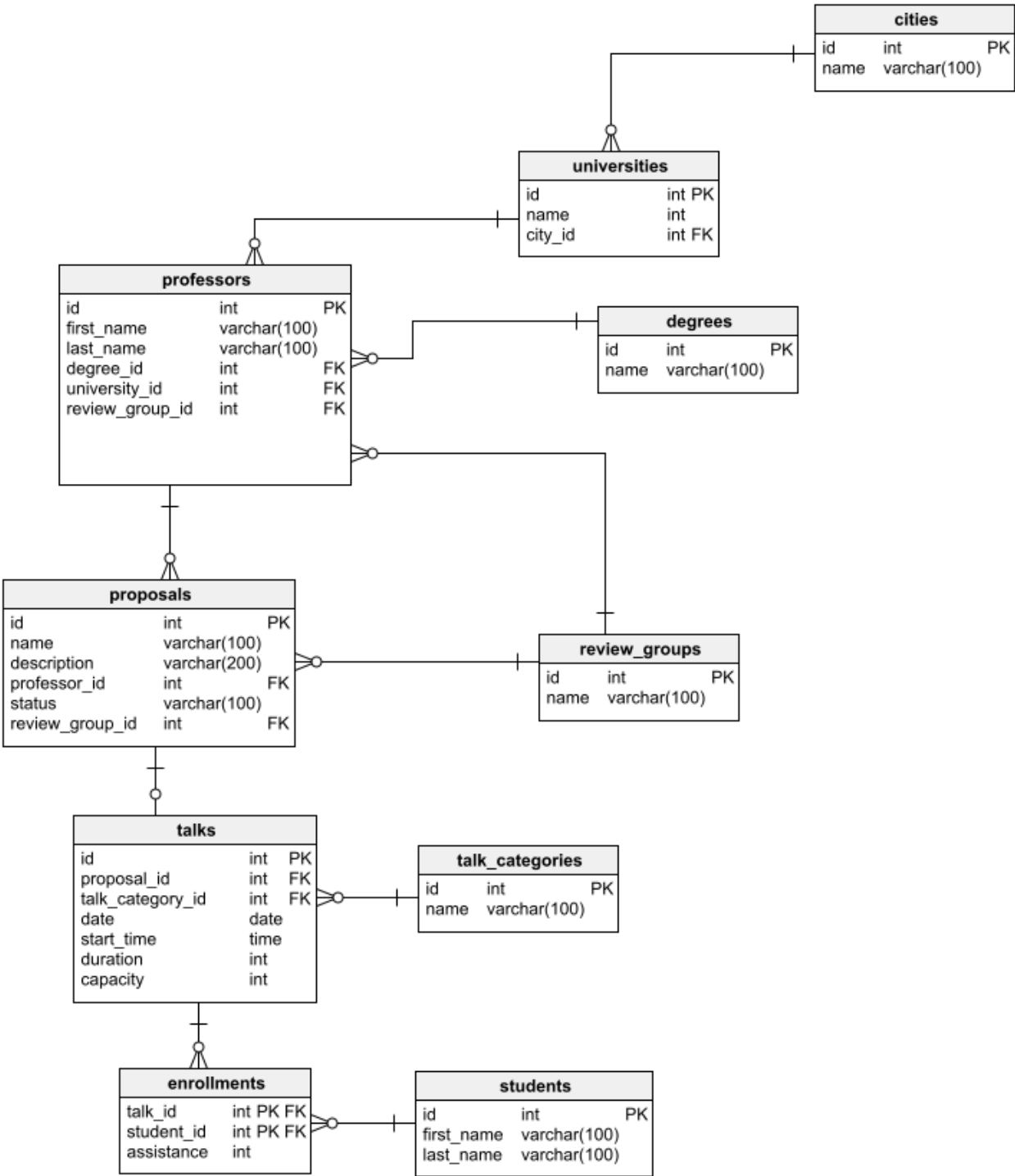
- El modelo físico de base de datos puede ser realizado mediante la herramienta de su preferencia.
- La solución también puede ser realizada a mano, sin embargo, deberá utilizar letra legible y gráficos debidamente detallados; las imágenes adjuntas deben ser nítidas.

Rúbrica de calificación

Criterio de calificación	Excelente	Promedio	Deficiente
C01. Modelado	Diseña el modelo físico de base de datos identifica las principales tablas y sus atributos, identifica las principales relaciones entre las tablas, las llaves primarias y las llaves secundarias.	Diseña el modelo físico de base de base de datos, identifica las principales tablas y atributos de la misma, sin embargo, identifica parcialmente las principales relaciones entre las tablas, las llaves primarias y las llaves secundarias.	No diseña el modelo físico de base de datos.
	6.0 puntos	3.0 punto	0 puntos
C02. Normalización	Aplica las formas normales 1FN, 2FN y 3FN para el modelo físico de base de datos realizado. Explica de forma clara la aplicación de las formas normales.	Aplica parcialmente las formas normales 1FN, 2FN y 3FN para el modelo físico de base de datos realizado o no explica de forma clara la aplicación de las formas normales.	No aplica las formas normales para el modelo físico de base de datos realizado.
	4.0 puntos	2.0 puntos	0 puntos
C03. DDL	Escribe las sentencias que permiten la creación de las bases de datos, las tablas y las relaciones entre tablas.	Escribe parcialmente las sentencias que permiten la creación de la base de datos, las tablas y las relaciones entre las tablas.	No escribe las sentencias que permiten la creación de la base de datos.
	2.0 puntos	1.0 puntos	0 puntos
C04. DML	Escribe las sentencias que permiten mostrar el nombre de la universidad con la mayor cantidad de propuestas recibidas. La consulta debe incluir las siguientes características: join de dos o más tablas, cálculo de funciones de agregación y anidación de consultas.	Escribe las sentencias que permiten mostrar el nombre de la universidad con la mayor cantidad de propuestas recibidas, sin embargo, no incluye alguna de las siguientes características: join de dos o más tablas, cálculo de funciones de agregación y anidación de consultas.	No escribe las sentencias que permiten mostrar el nombre de la universidad con la mayor cantidad de propuestas recibidas.
	4.0 puntos	2.0 punto	0 puntos
C05. DML	Escribe las sentencias que permiten mostrar el nombre de la charla con la mayor cantidad de alumnos inscritos. La consulta debe incluir las siguientes características: join de dos o más tablas, cálculo de funciones de agregación y anidación de consultas.	Escribe las sentencias que permiten mostrar el nombre de la charla con la mayor cantidad de alumnos inscritos, sin embargo, no incluye alguna de las siguientes características: join de dos o más tablas, cálculo de funciones de agregación y anidación de consultas.	No escribe las sentencias que permiten mostrar el nombre de la charla con la mayor cantidad de alumnos inscritos.
	4.0 puntos	2.0 punto	0 puntos
Total	20.0 puntos	10.0 puntos	0 puntos

Solucionario:

Modelado



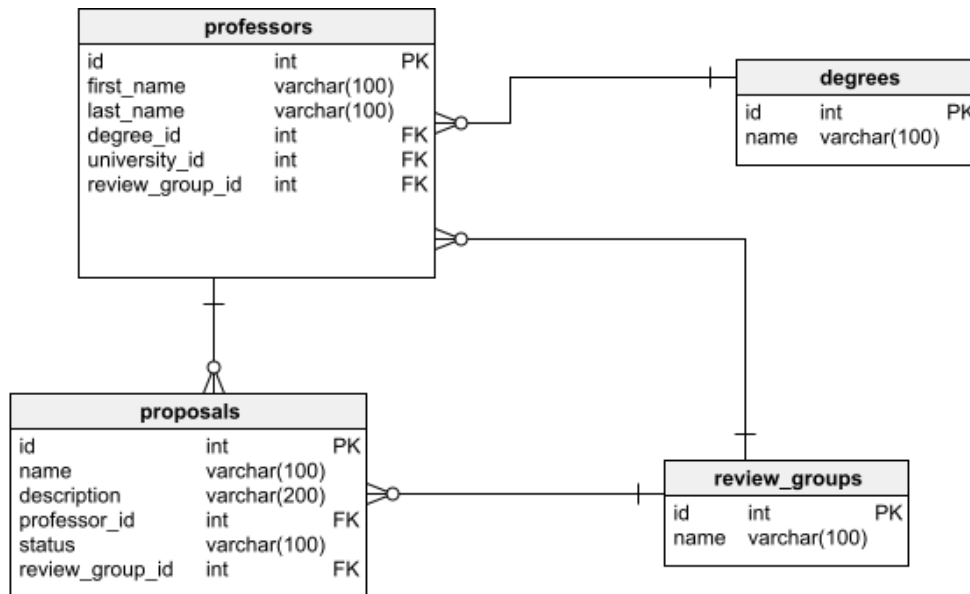
Normalización

1FN: No existen grupos repetitivos

Sustento: A fin de cumplir con la primera forma normal, se ha considerado tener la siguiente relación entre las tablas *professors* y *review_groups*.

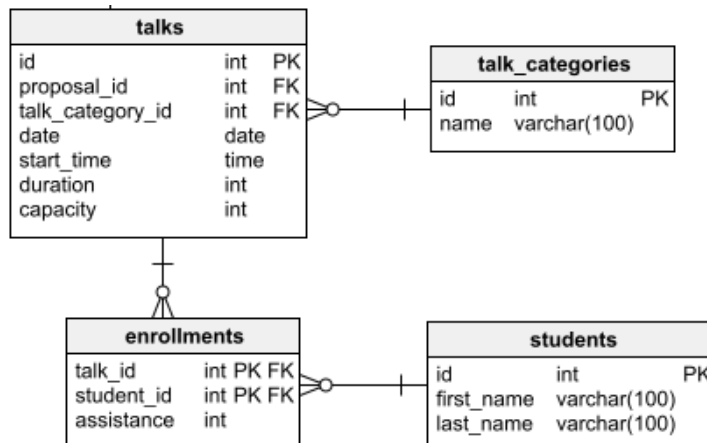
Un docente puede conformar un grupo revisor.

Un grupo revisor está conformado por varios docentes.



2FN: Solo deben haber dependencia funcionales completas.

Sustento: En la tabla *enrollments* se tiene una llave primaria compuesta por dos columnas. El campo asistencia depende tanto del alumno como la charla a la cual está inscrito.



3FN: No deben haber dependencias transitivas.

Sustento: En cada una de las tablas las columnas que no conforman parte de la llave primaria dependen directamente de la llave primaria y no de otra columna. En la tabla *talks* todas las columnas que no son parte de la llave primaria dependen de la columna *id*.

talks		
id	int	PK
proposal_id	int	FK
talk_category_id	int	FK
date	date	
start_time	time	
duration	int	
capacity	int	

DDL

-- Database: ea

```
CREATE DATABASE ea;  
GO;
```

use ea;

-- Table: cities

```
CREATE TABLE cities (  
    id int NOT NULL,  
    name varchar(100) NOT NULL,  
    CONSTRAINT cities_pk PRIMARY KEY (id)  
);
```

-- Table: degrees

```
CREATE TABLE degrees (  
    id int NOT NULL,  
    name varchar(100) NOT NULL,  
    CONSTRAINT degrees_pk PRIMARY KEY (id)  
);
```

-- Table: enrollments

```
CREATE TABLE enrollments (  
    talk_id int NOT NULL,  
    student_id int NOT NULL,  
    assistance int NOT NULL,  
    CONSTRAINT enrollments_pk PRIMARY KEY (talk_id,student_id)  
);
```

-- Table: professors

```
CREATE TABLE professors (  
    id int NOT NULL,  
    first_name varchar(100) NOT NULL,  
    last_name varchar(100) NOT NULL,  
    degree_id int NOT NULL,  
    university_id int NOT NULL,  
    review_group_id int NOT NULL,  
    CONSTRAINT professors_pk PRIMARY KEY (id)  
);
```

-- Table: proposals

```
CREATE TABLE proposals (  
    id int NOT NULL,  
    name varchar(100) NOT NULL,  
    description varchar(200) NOT NULL,
```

```

    professor_id int NOT NULL,
    status varchar(100) NOT NULL,
    review_group_id int NOT NULL,
    CONSTRAINT proposals_pk PRIMARY KEY (id)
);

-- Table: review_groups
CREATE TABLE review_groups (
    id int NOT NULL,
    name varchar(100) NOT NULL,
    CONSTRAINT review_groups_pk PRIMARY KEY (id)
);

-- Table: students
CREATE TABLE students (
    id int NOT NULL,
    first_name varchar(100) NOT NULL,
    last_name varchar(100) NOT NULL,
    CONSTRAINT students_pk PRIMARY KEY (id)
);

-- Table: talk_categories
CREATE TABLE talk_categories (
    id int NOT NULL,
    name varchar(100) NOT NULL,
    CONSTRAINT talk_categories_pk PRIMARY KEY (id)
);

-- Table: talks
CREATE TABLE talks (
    id int NOT NULL,
    proposal_id int NOT NULL,
    talk_category_id int NOT NULL,
    date date NOT NULL,
    start_time time NOT NULL,
    duration int NOT NULL,
    capacity int NOT NULL,
    CONSTRAINT talks_pk PRIMARY KEY (id)
);

-- Table: universities
CREATE TABLE universities (
    id int NOT NULL,
    name int NOT NULL,
    city_id int NOT NULL,
    CONSTRAINT universities_pk PRIMARY KEY (id)
);

```

DML

- Mostrar el nombre de la universidad con la mayor cantidad de propuestas recibidas.

```
select name
from (select u.name, count(p.id) quantity
      from proposals p
      join professors po on p.professor_id = po.id
      join universities u on po.university_id = u.id
      group by u.name) university_proposals
where quantity = (
  select max(quantity)
  from (select u.name, count(p.id) quantity
        from proposals p
        join professors po on p.professor_id = po.id
        join universities u on po.university_id = u.id
        group by u.name) university_proposals)
```

- Mostrar el nombre de la charla con la mayor cantidad de alumnos inscritos.

```
select name
from (select p.name, count(p.id) quantity
      from proposals p
      join professors po on p.professor_id = po.id
      join universities u on po.university_id = u.id
      join talks t on p.id = t.proposal_id
      group by p.name) university_proposals
where quantity = (
  select max(quantity)
  from (select p.name, count(p.id) quantity
        from proposals p
        join professors po on p.professor_id = po.id
        join universities u on po.university_id = u.id
        join talks t on p.id = t.proposal_id
        group by p.name) university_proposals)
```